Instructivo de Práctica — Sesión 7: Persistencia local con Hive

> 📌 **Qué vas a entregar:** al terminar esta práctica, tus «Gastos del viaje» (del backend de gastos) se van a poder leer sin conexión desde una caché de Hive propia de tu usuario, que se borra al cerrar sesión; y tus favoritos de lugares van a sobrevivir cerrar y volver a abrir la app. No hay tarea nueva asignada esta sesión — este trabajo se integra directo al Proyecto Final.

Objetivo

Entender cómo `GastosRepository` combina el backend de gastos (HTTP) con una caché local de Hive separada por usuario, conectar ese repositorio al `GastosController` (GetX) de la Sesión 6, y conservar los favoritos de lugares de ExploraEC en una caja propia. Distinguir qué conviene persistir, qué no (el token) y qué no puede resolver el almacenamiento local (crear datos sin red).

Requisitos previos

- Haber completado las Sesiones 4 a 6: `GetMaterialApp` con `Obx`, mapa y lugares reales, y el consumo del backend de gastos de la Sesión 6 (`GastosApiService` , `GastosController` , inicio de sesión y lista «Gastos del viaje» con sus tres estados y errores mapeados a `ApiException`).
- El backend de gastos corriendo en tu máquina o en la del docente (`docker compose up` , ver `backend-gastos-referencia.md`) y **tu propio usuario** ya registrado (Sesión 6), con al menos 2-3 gastos creados (por ejemplo desde `http://localhost:8000/docs`).
- Para el Paso 5: un **segundo usuario** de prueba (`POST /usuarios/`).
- Remoto `starter` ya agregado a tu proyecto `exploraec` (Sesión 2).

Tiempo estimado

Total: 60 minutos

Paso	Descripción	Tiempo
0	Recap de la Sesión 6: el problema de hoy	5 min
1	Traer el código de la Sesión 7 y explorar lo que ya viene resuelto	10 min
2	Conectar `GastosController` a `GastosRepository`	13 min
3	Favoritos de lugares persistentes	10 min
4	Verificar el comportamiento sin conexión	10 min
5	Aplicar la teoría: qué persiste y qué no	12 min
6 *(opcional)*	Idioma guardado con Hive — no cuenta dentro de los 60 min	10 min
Cierre	Guardar tu práctica: commit y push de `exploraec` — no cuenta dentro de los 60 min	3 min

---

Paso 0 — Recap de la Sesión 6: el problema de hoy (5 min)

Por qué se hace

Antes de traer código nuevo conviene sentir en tu propia app el problema que hoy se resuelve: tus gastos llegan del backend por la red, y cuando la red (o el servidor) falla no queda nada que mostrar. Este paso se hace **con tu proyecto tal como quedó en la Sesión 6**, antes de traer la rama de hoy.

Instrucciones

1. En tu proyecto `exploraec`, confirmar que el trabajo de la Sesión 6 está guardado:

```
git status
```

Si aparecen cambios sin guardar, hacer `git add -A && git commit -m "Sesión 6"` (el Punto de partida de abajo sobrescribe `lib/`). Antes de commitear, confirmar que `.timeout(...)` de `GastosApiService` vale otra vez `Duration(seconds: 15)` si lo cambiaste en la práctica de la Sesión 6. 2. Con el backend encendido (`curl http://localhost:8000/health` responde `{"status":"ok",...}`), ejecutar la app en el emulador (el mismo que usaste en la Sesión 6; si

está apagado, enciéndelo antes con `flutter emulators --launch <id-del-emulador>`), abrir la pestaña **Gastos**, iniciar sesión con tu correo y tu contraseña de la Sesión 6 y esperar la lista: se ven tus gastos.

```
flutter run
```

3. Detener el backend y forzar una recarga (en la pestaña Gastos, toca el ícono de recargar de la barra superior):

```
docker compose stop backend
```

Ejecutarlo en la carpeta del repositorio del backend (`proyecto-curso-spec-kit`); si el backend lo corre el docente, apagar en su lugar la red del emulador con `adb shell svc wifi disable` y `adb shell svc data disable`. Aparece el `ErrorView` de "Sin conexión": los gastos que ya estaban en pantalla se perdieron de la vista. Volver a encender el backend con `docker compose start backend` (o la red con `adb shell svc wifi enable` y `adb shell svc data enable`). 4. Cerrar la app por completo (detener `flutter run` con `q` y deslizar la app fuera de las apps recientes) y volver a abrirla desde el ícono del emulador: te pide iniciar sesión de nuevo (el token vive solo en memoria, Sesión 6) y todo se vuelve a pedir al servidor. Responder en una frase: ¿qué guardaría la app para no quedarse con las manos vacías sin conexión? ¿Y qué **no** debería guardar?

Resultado esperado

Se puede explicar que hoy falta una copia local de los últimos gastos (una caché) y un lugar donde guardar decisiones del usuario (favoritos de lugares) que sobrevivan al cierre de la app. Es lo que resuelve Hive. También que el token no es un dato para guardar sin protección (se resuelve en la Sesión 8).

Errores comunes

- **`git status` muestra cambios de la Sesión 6 sin guardar**. *Solución:* guardarlos con un commit antes de seguir: el Punto de partida sobrescribe `lib/`.
- **`adb: command not found`**. *Solución:* `adb` vive en `platform-tools` dentro del SDK de Android (por ejemplo `~/Android/Sdk/platform-tools`); agrégalo al PATH o usa el interruptor de modo avión del emulador.
- **`docker compose stop backend` dice `no configuration file provided`**. *Solución:* ejecutarlo dentro de la carpeta del repositorio del backend, donde está `docker-compose.yml`.

✅ Antes de continuar, confirma que...

- ☐ El trabajo de la Sesión 6 está guardado en un commit.
- ☐ Viste el `ErrorView` de "sin conexión" con el backend detenido (o la red apagada), y todo quedó encendido otra vez.
- ☐ Puedes decir qué datos convendría guardar en el dispositivo y cuál no (el token).

Punto de partida

```
git fetch starter
git checkout starter/sesion-07 -- lib test pubspec.yaml PLACEHOLDERS.md
flutter pub get
```

Esto trae, ya escritos y completos, `lib/repositories/gastos_repository.dart`, `lib/models/gasto.dart` y `lib/models/place.dart` (con `toMap()` / `fromMap()`), `lib/services/gastos_api_service.dart` (con el método nuevo `obtenerIdUsuario()`), `lib/controllers/gastos_controller.dart` (con el repositorio ya declarado y abierto al iniciar sesión), `lib/screens/favorites_screen.dart`, `lib/main.dart` (con Hive ya inicializado) y `lib/services/settings_service.dart` (para el Paso 6 opcional) — además de `hive`, `hive_flutter` y `path_provider` en `pubspec.yaml`. También trae `test/gastos_controller_test.dart` (10 pruebas del controller con un servidor falso y una carpeta temporal de Hive: **no** necesitan el backend ni el emulador); las usarás como comprobación en los Pasos 1, 2 y 5. Ver `PLACEHOLDERS.md` para el detalle exacto.

`git checkout` no borra archivos que la rama ya no tiene: `lib/screens/favorites_placeholder_screen.dart` (el placeholder de la Sesión 2) queda en tu proyecto sin usarse. No molesta; si quieres, bórralo:

```
rm lib/screens/favorites_placeholder_screen.dart
```

Paso 1 — Explorar lo que ya viene resuelto (10 min)

Por qué se hace

Hive necesita inicializarse antes de `runApp` (por eso `main()` es `async`), y un repositorio que

combina servidor + caché por usuario es una pieza nueva conceptualmente — conviene leerlo con calma antes de conectar nada, en vez de escribirlo desde cero bajo presión de tiempo.

Instrucciones

1. Con los archivos ya traídos a tu proyecto `exploraec` (Punto de partida), abrir `lib/main.dart`: identificar por qué `main()` es `async`, y el orden exacto `WidgetsFlutterBinding.ensureInitialized()` → `await Hive.initFlutter()` → `await Hive.openBox<Map>('favoritos')` → `runApp(...)`. Notar que **no** se abre ninguna caja de gastos aquí: esa caja depende de quién inicie sesión.
2. Abrir `lib/models/gasto.dart` y ubicar `toMap()` / `Gasto.fromMap(...)`: confirmar que `Gasto.fromMap(unGasto.toMap())` reconstruye el mismo objeto (los 5 campos — `id`, `descripcion`, `monto`, `categoria`, `fecha` — son primitivos, sin necesitar código generado). Mismos nombres de campo que el JSON del backend.
3. Abrir `lib/repositories/gastos_repository.dart` y leerlo completo: identificar que `obtenerGastos()` intenta `GastosApiService` primero, guarda en caché si tiene éxito (junto con la fecha de sincronización), y si falla por **red o servidor** (sin conexión, tiempo agotado, 5xx) lee la caché y devuelve un **record** `(List<Gasto>, bool)` — el segundo valor indica si el dato viene de respaldo. Identificar también qué errores **no** usan la caché (401 sesión caducada, 400/403/404) y por qué.
4. En el mismo archivo, ubicar `abrirParaUsuario(...)`: abre la caja `gastos_<id>`, donde `<id>` sale de `GET /usuarios/me`. Cada usuario tiene su propia caja. Ubicar también `vaciar()` (se usa en el Paso 5).
5. Abrir `lib/controllers/gastos_controller.dart`: confirmar que ya declara `_repository = GastosRepository(_api)` (el mismo `GastosApiService` de la Sesión 6) y que `entrar(...)` llama a `_repository.abrirParaUsuario()` justo después del login. Buscar en `lib/` todo lo que usa Hive y confirmar que **ningún** archivo guarda el token ahí (el token sigue siendo el campo `_token` de `gastos_api_service.dart`):

```
grep -rn "Hive\." lib/
```

Deben aparecer solo `main.dart` (inicialización y caja `favoritos`), `gastos_repository.dart` (caja `gastos_<id>`), `places_controller.dart` (caja `favoritos`) y `settings_service.dart` (Paso 6). 6. Ejecutar la app y confirmar que arranca sin errores de Hive en la consola (todavía se ve igual que al final de la Sesión 6 — nada de esto se conecta hasta el Paso 2). La primera ejecución tarda más de lo normal porque compila los plugins nuevos (`path_provider`):

```
flutter run
```

7. Sin detener la app, abrir **otra terminal** en tu proyecto `exploraec` y ejecutar las pruebas que trajo el Punto de partida:

```
flutter test
```

Con la rama recién traída deben terminar **7 pasando y 3 fallando**, a propósito: las tres que fallan se llaman «Paso 2 — sin conexión con caché…», «Paso 2 — 5xx con caché…» y «Paso 5 — cerrar sesión borra la caja del usuario», porque esas conexiones las haces tú en los Pasos 2 y 5. Si las 3 fallan por otro motivo (error de compilación), revisa `flutter pub get`.

Resultado esperado

Se puede explicar, en una frase cada una: por qué `main()` es `async`, qué hace `toMap()` / `fromMap()`, qué hace `GastosRepository.obtenerGastos()` en éxito y en falla, y por qué la caja se llama `gastos_<id>` y no simplemente `gastos`. La app arranca sin errores.

Errores comunes

- **`HiveError: You need to initialize Hive or provide a path`**. *Solución:* revisar que el `pubspec.yaml` traído en el Punto de partida incluye `hive` / `hive_flutter` / `path_provider` y que se corrió `flutter pub get` — si el error persiste, confirmar que no se mezcló `main.dart` con una versión de otra sesión.
- **`Target of URI doesn't exist: 'package:hive_flutter/hive_flutter.dart'`**. *Solución:* falta `flutter pub get` tras traer el `pubspec.yaml`.

✅ Antes de continuar, confirma que...

- ☐ La app arranca sin errores de Hive en la consola y `flutter test` da 7 pruebas pasando y 3 fallando (las de los Pasos 2 y 5).
- ☐ Puedes explicar qué hace `GastosRepository.obtenerGastos()` cuando hay servidor y cuando no lo hay.
- ☐ Confirmaste que el token no se guarda en Hive.

Paso 2 — Conectar `GastosController` al repositorio (13 min)

Por qué se hace

Con el repositorio ya construido, el controller deja de hablarle directamente a `GastosApiService` — pasa a pedirle todo al repositorio, ganando el respaldo sin conexión sin cambiar su propia lógica de estados (los mismos tres de la Sesión 6).

Instrucciones

1. En `lib/controllers/gastos_controller.dart`, ubicar el bloque `TODO(sesion-07)` de `cargarGastos()` y su comentario `// Por qué:`.
2. Borrar el bloque que llama a `_api.listarGastos(...)` directamente (desde `final resultado = ...` hasta `estado.value = EstadoCarga.exito;`, incluidas las líneas `totalEnServidor` y `desdeCache.value = false;`) y descomentar el bloque que llama a `_repository.obtenerGastos()`, que además guarda `desdeCache.value`. **No** borres ningún `import`: `_api` se sigue usando para registrar, iniciar sesión y cerrar sesión. El `catch (ApiException e)` que ya muestra el `ErrorView` no cambia. El comentario `TODO(sesion-07)` y su `// Por qué:` **se quedan** (como en la Sesión 6): lo que desaparece es el código provisional, y el bloque nuevo queda sin `//`.
3. En `lib/screens/gastos_screen.dart`, dentro del `Obx`, reemplazar la línea final `return _buildLista(controller.gastos);` por una columna con el aviso y la lista:

```
        return Column(
          children: [
            if (controller.desdeCache.value)
              MaterialBanner(
                content: Text(
                  'Sin conexión — mostrando tus gastos guardados '
                  '(última sincronización: ${controller.ultimaSincronizacion})',
                ),
                leading: const Icon(Icons.cloud_off),
                actions: [
                  TextButton(
                    onPressed: controller.cargarGastos,
                    child: const Text('Reintentar'),
                  ),
                ],
              ),
            Expanded(child: _buildLista(controller.gastos)),
          ],
        );
```

(Las líneas comentadas que quedan debajo, `// return RefreshIndicator(...)`, son el Paso 10 opcional de la Sesión 6: déjalas como están.) 4. Guardar y hacer hot reload (`r` en la terminal de `flutter run`), iniciar sesión y abrir «Gastos del viaje». 5. En esa misma terminal nueva (en tu proyecto `exploraec`) ejecutar las pruebas:

```
flutter test
```

Ahora pasan **9** y falla **1**: «Paso 5 — cerrar sesión borra la caja del usuario» (se arregla en el Paso 5). Las dos de «Paso 2» ya pasan.

Resultado esperado

Con el backend encendido, la app funciona igual que en la Sesión 6 (sin ningún banner). Sin conexión y con algo ya cacheado, aparece el banner con la hora de la última sincronización y la lista muestra los gastos guardados.

Errores comunes

- **El banner se queda pegado incluso después de recuperar la conexión**. *Solución:* confirmar que `desdeCache.value` se pone en `false` explícitamente en el camino de éxito

real, no solo en `true` en el camino de respaldo.

- **`type '_Map<dynamic, dynamic>' is not a subtype of type 'Map<String, dynamic>'`**. *Solución:* Hive devuelve mapas con claves `dynamic`; `Gasto.fromMap` debe convertirlos con `Map<String, dynamic>.from(...)` (ya viene así en `gasto.dart`; revisar que no se sobrescribió con otra versión).
- **Con el backend detenido sale el `ErrorView` en vez del banner, aun habiendo cargado antes**. *Solución:* la caja se abre al iniciar sesión (`abrirParaUsuario`); si se inició sesión antes de traer la rama de hoy, cerrar la app, volver a abrirla e iniciar sesión de nuevo con el backend encendido para que se guarde la primera copia.

✅ Antes de continuar, confirma que...

- ☐ `cargarGastos()` usa `_repository`, no `_api.listarGastos` directamente (sin código comentado debajo del `TODO`), `flutter analyze` termina con `No issues found!` y `flutter test` da 9 pasando y 1 fallando (la del Paso 5).
- ☐ Existe un banner visible cuando `desdeCache` es `true`.
- ☐ En `gastos_controller.dart` solo queda pendiente el `TODO(sesion-07)` de `salir()` (Paso 5); el de `cargarGastos()` ya está resuelto.

Paso 3 — Favoritos de lugares persistentes (10 min)

Por qué se hace

ExploraEC conserva sus lugares y su mapa (Sesiones 1-5), y un favorito de lugar es una decisión del usuario que debe sobrevivir tanto reiniciar la app como que la lista de lugares cercanos cambie por completo — por eso vive en su propia caja de Hive (`favoritos`), independiente de la caché de gastos del Paso 2. Aquí los favoritos son locales al dispositivo y **no** se separan por usuario: son lugares públicos, no datos personales como los gastos (ver Paso 5).

Instrucciones

1. En `lib/controllers/places_controller.dart`, ubicar el bloque `TODO(sesion-07)` (es el único del archivo) y su comentario `// Por qué:`.
2. Borrar la versión en memoria de `alternarFavorito` (la de la Sesión 4, que solo agrega o quita de la lista `favoritos`) y descomentar el cuerpo real: agrega/quita de `_favoritosBox` (con `toMap()` / `fromMap()`) **y** del `RxList favoritos`. El `TODO` y su `// Por qué:` se quedan.

3. Confirmar que `onInit()` ya carga `favoritos` desde `_favoritosBox` al arrancar (código ya completo, revisarlo sin modificarlo).
4. Guardar, hacer hot reload (`r`), marcar 2-3 lugares como favoritos desde el ícono de corazón en `PlaceCard` (ya trae el `Obx` conectado, Punto de partida), y revisar la pestaña Favoritos (`FavoritesScreen` , ya trae la pantalla completa).

Resultado esperado

Los lugares marcados aparecen en la pestaña Favoritos, con el mismo diseño de tarjeta (`PlaceCard`) que Inicio. El ícono de corazón cambia de vacío a relleno al tocarlo.

Errores comunes

- **El corazón no cambia de estado al tocarlo**. *Solución:* confirmar que se descomentó el cuerpo completo de `alternarFavorito` (no solo parte de él) — `place_card.dart` ya trae el `Obx` correcto desde el Punto de partida, el problema casi siempre está en el controller.
- **Un favorito marcado desaparece al reiniciar la app**. *Solución:* revisar que `alternarFavorito` realmente escribe en `_favoritosBox` (no solo en el `RxList` en memoria) — sin esa escritura, el dato no sobrevive cerrar la app. Es justo el comportamiento de la versión en memoria de la Sesión 4 (Paso 6), y la razón de este paso.

✅ Antes de continuar, confirma que...

- ☐ Marcar/desmarcar un favorito actualiza el ícono de inmediato.
- ☐ La pestaña Favoritos muestra exactamente los lugares marcados.
- ☐ `alternarFavorito` escribe en `_favoritosBox` y no queda código comentado debajo del `TODO(sesion-07)` de `places_controller.dart` .

Paso 4 — Verificar el comportamiento sin conexión (10 min)

Por qué se hace

El código puede compilar perfecto y aun así fallar en el escenario real que motivó toda la sesión — solo probarlo con el servidor inalcanzable a propósito confirma que el respaldo funciona.

Instrucciones

1. Con la app corriendo, la sesión iniciada y «Gastos del viaje» ya cargado **con el backend encendido** (así la caché queda guardada), detén el backend (o apaga la red del emulador / activa el modo avión en un dispositivo físico). En la terminal A, dentro de la carpeta `proyecto-curso-spec-kit`:

```
docker compose stop backend
```

Con el servidor apagado la conexión se rechaza al instante. Apagar la red del emulador (`adb shell svc wifi disable` y `adb shell svc data disable`) también falla de inmediato en el emulador Pixel_10_Pro; para volver a encenderla, `adb shell svc wifi enable` y `adb shell svc data enable`. 2. Reinicia la carga tocando el ícono de recargar de la barra superior de Gastos: debe verse la lista guardada con el banner de "sin conexión, gastos guardados" y la hora de la última sincronización. 3. Reactiva el servicio (`docker compose start backend` en la terminal A, o la red del emulador), espera a que `curl http://localhost:8000/health` responda, toca "Reintentar" en el banner y confirma que el banner desaparece en la siguiente carga exitosa. 4. Cierra por completo la app (detén `flutter run` con `q` y desliza la app fuera de las apps recientes) y ábrela de nuevo desde el ícono del emulador: los favoritos de lugares marcados en el Paso 3 siguen ahí. Observa también que **tus gastos piden iniciar sesión otra vez**: el token solo vive en memoria. Con el backend apagado, ese inicio de sesión fallaría, así que la caché solo ayuda con la sesión ya abierta. Ese límite se resuelve en la Sesión 8. Deja la app cerrada: en el Paso 5 se vuelve a ejecutar con `flutter run`.

Resultado esperado

El banner aparece exactamente cuando el servidor no responde y hay caché, y desaparece cuando vuelve. Los favoritos sobreviven cerrar y reabrir la app. Sin sesión iniciada no se puede llegar a la caché de gastos.

Errores comunes

- **El emulador tarda en reflejar el cambio de red**. *Solución:* esperar unos segundos después de desactivar el Wi-Fi y los datos antes de recargar. Detener el backend con Docker es inmediato y es el método más fiable.
- **Apagué el Wi-Fi del emulador y la app sigue cargando datos**. *Solución:* según la versión del emulador puede seguir alcanzando `10.0.2.2` (la máquina anfitriona) aunque el Wi-Fi virtual esté apagado. Usa `docker compose stop backend`, que corta la conexión de forma verificable.
- **Con el backend apagado y la app recién abierta no hay lista, solo la pantalla de inicio de sesión**. *Solución:* es el comportamiento esperado: sin token no hay llamada ni

caja abierta. Inicia sesión con el backend encendido, carga una vez y luego apágalo.

- **Aparece "sesión caducada" en vez del banner**. *Solución:* el token dura 30 minutos (`ACCESS_TOKEN_EXPIRE_MINUTES`). Un `401` no usa la caché a propósito: vuelve a iniciar sesión.
- **Con el backend encendido el banner aparece igual**. *Solución:* el repositorio usa la caché ante cualquier falla de red o 5xx. Revisar `docker compose ps` y `curl http://localhost:8000/health`, y tocar "Reintentar".

✅ Antes de continuar, confirma que...

- ☐ Se comprobó el banner de "sin conexión, gastos guardados" apagando el backend (o la red) a propósito.
- ☐ Los favoritos sobrevivieron cerrar y reabrir la app.
- ☐ Sabes por qué al reabrir la app hay que volver a iniciar sesión.

Paso 5 — Aplicar la teoría: qué persiste y qué no (12 min)

Por qué se hace

La teoría de hoy separó cosas que se parecen pero no son lo mismo: la **caché** (una copia de lo que devolvió el servidor, desechable y **de un usuario**), los **favoritos** (una decisión del usuario sobre lugares públicos), el **token** (un secreto que no va a Hive sin protección) y lo que **no existe en el servidor** (un gasto creado sin red). Este paso lo comprueba en tu propia app.

Instrucciones

1. En `lib/controllers/gastos_controller.dart`, ubicar el bloque `TODO(sesion-07)` de `salir()` (el método que ejecuta «Cerrar sesión») y su comentario `// Por qué:`. Borrar el método provisional (solo olvida el token en memoria) y descomentar el bloque que además llama a `_repository.vaciar()` (cierra y borra de disco la caja `gastos_<id>` del usuario). Como cerraste la app en el Paso 4, vuelve a ejecutarla en la terminal B con `flutter run` (si la tuvieras corriendo, basta hot reload con `r`).
2. Con el backend encendido, abrir la pestaña Gastos, iniciar sesión con **tu usuario**, abrir «Gastos del viaje» (queda en caché) y tocar «Cerrar sesión» (menú « ⋮ » de la barra superior de Gastos). Registrar a un segundo usuario si aún no existe:

```
curl -X POST http://localhost:8000/usuarios/ \
  -H "Content-Type: application/json" \
  -d '{"email":"segundo@ejemplo.com","password":"clave12345"}'
```

Iniciar sesión con el segundo usuario: su lista está vacía (no ve tus gastos) y tu caja ya no existe en el dispositivo. Volver a entrar con tu usuario: tus gastos vuelven desde el servidor (y se vuelve a crear tu caja).

(Opcional) Comprobar las cajas que hay en el almacenamiento de la app (solo funciona con la app en modo debug, la de `flutter run`); repítelo con tu usuario dentro, después de cerrar sesión y con el segundo usuario dentro:

```
adb shell run-as com.example.exploraec ls app_flutter
```

Con una sesión abierta aparece un archivo `gastos_<id>.hive` (el `<id>` de ese usuario; con tu usuario de la Sesión 6 suele ser `gastos_1.hive`) junto a `favoritos.hive`, más archivos `.lock` que puedes ignorar; tras «Cerrar sesión» el de gastos desaparece y `favoritos.hive` sigue ahí. 3. Comprobar los favoritos de lugares: con conexión, tocar "+" en Inicio, agregar un lugar propio (nombre "Mi lugar de prueba"; categoría y descripción, cualquier texto) y tocar «Guardar». Marcarlo como favorito (corazón; la tarjeta nueva queda al final de la lista, desplázate; el botón «+» puede tapar parte de su corazón: toca la mitad de arriba del corazón o desplaza un poco más) y confirmar que aparece en Favoritos. Cerrar la app por completo y volver a abrirla (e iniciar sesión): en Inicio **ya no está** (los lugares creados a mano viven solo en memoria); en Favoritos **sigue ahí** (caja propia de Hive, que no se borra al cerrar sesión). Para el Paso 6 opcional, vuelve a ejecutar `flutter run` en la terminal B. 4. Discusión en parejas (sin código): supón que, con el backend apagado, el turista intenta registrar «Almuerzo en el Mercado Central, 6.50». Ese gasto **no existe en el servidor**: la caché de hoy es solo de lectura y no lo guardaría. Responder por escrito en 3-4 líneas:

- ¿Dónde habría que guardarlo hasta tener red, y con qué estado (por ejemplo «pendiente de sincronizar»)?
- ¿Qué pasaría si el servidor lo rechaza después, por ejemplo con el error de límite de 500 por categoría?
- ¿Qué id tendría el gasto antes de que el servidor le asigne el suyo?

No se implementa hoy: es el diseño que se retoma en la Sesión 8. 5. Responder en una frase: ¿por qué la caja de gastos es por usuario y se vacía al cerrar sesión, mientras que los favoritos de lugares no? (Pista: privacidad en un teléfono compartido; Hive guarda los datos **sin cifrar** en el almacenamiento de la app.) 6. En una terminal nueva (en tu proyecto

`exploraec`; la app puede estar corriendo o no), ejecutar las pruebas otra vez:

```
flutter test
```

Deben pasar las **10** (`All tests passed!`): la del Paso 5 ya pasa porque `salir()` vacía la caja.

Resultado esperado

Tras cerrar sesión, el siguiente usuario no ve tus gastos. Un lugar propio desaparece de Inicio pero sigue en Favoritos, porque cada caja tiene un ciclo de vida distinto: la de gastos se reemplaza en cada carga exitosa y se borra al salir; la de favoritos solo cambia cuando el usuario lo decide. Se puede explicar por qué un gasto creado sin red necesita un estado «pendiente de sincronizar».

Errores comunes

- **Después de cerrar sesión, el segundo usuario ve los gastos del primero**. *Solución:* no se descomentó la llamada a `_repository.vaciar()`, o `abrirParaUsuario` abrió una caja con un nombre fijo en vez de `gastos_<id>`.
- **`HiveError: ... box ... is already open` / `deleteFromDisk` falla**. *Solución:* `vaciar()` debe cerrar la caja antes de borrarla (`await box.deleteFromDisk()` ya cierra la caja; no llamar a `close()` después sobre la misma instancia).
- **El lugar propio sigue apareciendo en Inicio tras reabrir la app**. *Solución:* no se cerró la app por completo (queda en segundo plano); deslizarla fuera de las apps recientes y volver a abrirla.
- **Tras reabrir, los favoritos aparecen en otro orden**. *Solución:* es normal: Hive devuelve las entradas por su clave, no por el orden en que las marcaste.
- **No se encuentra el lugar propio en Inicio**. *Solución:* se agrega al final de la lista; desplazarse hasta abajo.
- **`curl` responde `400` al registrar al segundo usuario**. *Solución:* el email ya existe (usa otro) o la contraseña no tiene entre 8 y 72 caracteres.

✅ Antes de continuar, confirma que...

- ☐ Un segundo usuario no ve los gastos del primero, y la caja del primero se borró al cerrar sesión.
- ☐ El lugar propio desapareció de Inicio pero sigue en Favoritos tras reabrir la app.
- ☐ Puedes explicar por qué la caché de gastos es por usuario y los favoritos no.

- ☐ Puedes explicar qué es un gasto «pendiente de sincronizar» y por qué hoy no se puede crear sin red.
- ☐ `flutter test` termina con `All tests passed!` (10 pruebas).

Paso 6 (opcional) — Idioma guardado con Hive (10 min)

> Este paso **no cuenta dentro de los 60 minutos** ni bloquea la Sesión 8. Hazlo si terminaste antes que el resto o como repaso. Necesita `flutter run` activo en la terminal B (hot restart con `R`). Sigue la misma mecánica de los pasos anteriores: leer el `TODO`, borrar lo que indica, descomentar el bloque y probarlo en la app.

Por qué se hace

En la Sesión 4, el idioma elegido con el botón de traducir se perdía al cerrar la app (era estado global, pero solo en memoria). Hoy ya hay una base de datos local: este paso guarda esa elección en **una caja de Hive** y la lee al arrancar. Es el caso más simple de persistencia: una caja de clave-valor (`'idioma'` → `'es'` o `'en'`), sin adaptadores ni serialización, porque el valor ya es un `String`. El archivo `lib/services/settings_service.dart` viene completo; solo falta conectarlo.

Instrucciones

1. En `lib/main.dart`, descomentar el `import 'services/settings_service.dart';` de la parte superior y, dentro de `main()`, la línea `await SettingsService.abrir();` (justo debajo de las dos cajas que ya se abren). No hay nada que borrar.
2. En el mismo archivo, dentro de `GetMaterialApp`, borrar la línea `locale: const Locale('es', 'EC'),` y descomentar `locale: SettingsService.idioma,`. Leer antes el comentario `// Por qué:`.
3. En `lib/screens/home_screen.dart`, descomentar el `import '../services/settings_service.dart';` (no hay nada que borrar).
4. En el mismo archivo, en el `IconButton` de idioma, borrar el bloque `onPressed: () { ... },` y descomentar `onPressed: SettingsService.alternarIdioma,`.
5. Hacer hot restart (`R`; cambió `main()`). Tocar el ícono de traducir: la barra inferior pasa a "Home / Map / Favorites / Expenses".
6. Hacer hot restart (`R`) otra vez, o detener la app y volver a ejecutarla: **la app arranca en inglés**. Tocar el ícono de nuevo para volver a español y comprobar que también se

recuerda.

Resultado esperado

El idioma elegido se conserva entre ejecuciones. La primera vez (sin nada guardado) la app arranca en español.

Errores comunes

- `HiveError: Box not found. Did you forget to call Hive.openBox()?` al arrancar. *Solución:* se descomentó `locale: SettingsService.idioma` pero no `await SettingsService.abrir();` en `main()`. Hive solo deja leer una caja que ya está abierta.
- `Undefined name 'SettingsService'`. *Solución:* falta descomentar el `import` en ese archivo (hay uno en `main.dart` y otro en `home_screen.dart`).
- `Duplicated named argument 'locale'` o el botón sigue sin recordar el idioma. *Solución:* quedaron las dos versiones; borrar la línea `locale: const Locale('es', 'EC'),` y el bloque `onPressed: () { ... }` viejos, no solo comentarlos.
- **El idioma cambia pero al reiniciar vuelve a español**. *Solución:* el botón sigue usando el `onPressed` viejo (que cambia el idioma sin guardarlo). Usar `SettingsService.alternarIdioma`.
- **Siguen apareciendo claves como `inicio`**. *Solución:* hot reload no basta con cambios en `main()`; hacer hot restart (`R`).

✅ Antes de continuar, confirma que...

- ☐ Al cambiar a inglés y reiniciar la app, sigue en inglés.
- ☐ Puedes explicar por qué la caja `ajustes` se abre en `main()` antes de `runApp`.

Para pensar *(sin solución escrita)*

Para un solo valor simple como este, ¿qué otra alternativa de las vistas en la teoría (clave-valor) serviría? ¿Por qué aquí conviene reutilizar Hive, que ya está en el proyecto?

Cierre — Guardar tu práctica: commit y push de `exploraec` (3 min)

Por qué se hace

Como en la Sesión 6, al final se guarda el trabajo en tu repositorio de GitHub: así queda respaldado y la Sesión 8 parte de un estado conocido (su Paso 0 pide tener la Sesión 7 guardada en un commit). Se guarda **solo la app** (`exploraec`); el backend es otro repositorio

y no se toca.

Instrucciones

En la **terminal B**, dentro de tu proyecto:

```
cd exploraec-curso/exploraec
flutter analyze
flutter test
git status --short
git add -A
git commit -m "Sesión 7: caché de gastos con Hive y favoritos persistentes"
git push
git log --oneline -3
git status -sb
```

`flutter analyze` debe terminar con `No issues found!` y `flutter test` con `All tests passed!`. `git status --short` lista los archivos de `lib/`, `test/gastos_controller_test.dart`, `pubspec.yaml`, `pubspec.lock` y `PLACEHOLDERS.md`; también pueden aparecer `linux/flutter/generated_plugins.cmake` y `windows/flutter/generated_plugins.cmake`: los modifica Flutter solo al agregar los plugins nuevos (`path_provider`) y es normal. **No** debe aparecer `proyecto-curso-spec-kit`.

Resultado esperado

`git push` termina mostrando algo como `3a66f8f..249fad3 main -> main`, `git log --oneline -3` muestra arriba tu commit «Sesión 7…» y `git status -sb` muestra `## main...origin/main` **sin** la palabra `ahead`.

Errores comunes

Los mismos del Cierre de la Sesión 6: `Author identity unknown` (configura `git config --global user.name` y `user.email`), `no upstream branch` (`git push -u origin main`), `rejected` (`git pull --rebase` y luego `git push`) y `nothing to commit` (ya estaba guardado: solo `git push`).

✅ Antes de terminar la sesión, confirma que...

- ☐ `git status -sb` muestra `## main...origin/main` sin `ahead`.
- ☐ Detuviste el backend (`docker compose stop`) si ya no lo necesitas.

Anexo — Todos los comandos de la práctica, en orden

Resumen para copiar y pegar (Linux/Mac). Las líneas `# [editar a mano]` son cambios de archivo explicados en el paso indicado. `<id-del-emulador>` es el que muestra `flutter emulators`.

Paso 0 (terminal B en `exploraec-curso/exploraec`; terminal A en `exploraec-curso/proyecto-curso-spec-kit`):

```
git status
git add -A && git commit -m "Sesión 6"        # solo si git status muestra cambios sin
 guardar
flutter emulators --launch <id-del-emulador>   # solo si el emulador está apagado
curl http://localhost:8000/health
flutter run
docker compose stop backend                     # terminal A; recargar en la app
docker compose start backend
# q en la terminal de flutter run; cerrar la app y abrirla desde el ícono
```

Punto de partida y Paso 1:

```
git fetch starter
git checkout starter/sesion-07 -- lib test pubspec.yaml PLACEHOLDERS.md
flutter pub get
rm lib/screens/favorites_placeholder_screen.dart   # opcional
grep -rn "Hive\." lib/
flutter run
flutter test                                       # 7 pasan, 3 fallan a propósito
```

Pasos 2 y 3:

```
# [editar a mano] lib/controllers/gastos_controller.dart: cargarGastos() (Paso 2,
punto 2)
# [editar a mano] lib/screens/gastos_screen.dart: Column con MaterialBanner (Paso 2,
punto 3)
# r en la terminal de flutter run
# [editar a mano] lib/controllers/places_controller.dart: alternarFavorito (Paso 3)
flutter analyze
# r en la terminal de flutter run
flutter test                                        # 9 pasan, 1 falla (Paso 5)
```

Paso 4 (terminal A):

```
docker compose stop backend
docker compose start backend
curl http://localhost:8000/health
# q en la terminal de flutter run; cerrar la app y abrirla desde el ícono
```

Paso 5:

```
# [editar a mano] lib/controllers/gastos_controller.dart: salir() (Paso 5, punto 1)
flutter run
curl -X POST http://localhost:8000/usuarios/ -H "Content-Type: application/json" -d
'{"email":"segundo@ejemplo.com","password":"clave12345"}'
adb shell run-as com.example.exploraec ls app_flutter
flutter test                                        # 10 pasan
```

Paso 6 (opcional):

```
# [editar a mano] lib/main.dart y lib/screens/home_screen.dart (Paso 6, puntos 1 a 4)
# R en la terminal de flutter run (hot restart), dos veces
```

Cierre:

```
flutter analyze
flutter test
git add -A
git commit -m "Sesión 7: caché de gastos con Hive y favoritos persistentes"
git push
```

Entregable de la práctica

- `GastosRepository` funcionando: servidor primero, caché de Hive por usuario como respaldo verificado con el backend detenido.
- Caja de gastos vaciada al cerrar sesión y comprobada con un segundo usuario (Paso 5).
- Favoritos de lugares persistentes, con `FavoritesScreen` real mostrando exactamente los lugares marcados.
- Diferencia comprobada entre caché (desechable) y favoritos (decisión del usuario) con un lugar propio (Paso 5), y respuesta escrita sobre el gasto «pendiente de sincronizar».
- `flutter test` con las 10 pruebas pasando.
- Token **no** guardado en Hive (verificado con `grep`); ningún dato "inventado": el banner de caché nunca se oculta cuando corresponde mostrarlo.
- *(Opcional)* Idioma guardado con Hive (Paso 6).
- Tu práctica guardada y subida a tu repositorio (commit y push del Cierre).

Conexión con la siguiente sesión

En la Sesión 8 la app pasa al backend propio de gastos con Dio: registro/login, el token JWT sale de la memoria y va a almacenamiento seguro (`flutter_secure_storage` ; hoy no se guarda en Hive a propósito), un interceptor lo adjunta, y el CRUD completo permite crear, editar y borrar gastos, incluidos los errores de negocio (el límite de 500 por categoría). Lo que hoy solo discutiste — qué hacer con un gasto creado sin red — se vuelve diseño y práctica. `GastosRepository` y `GastosController` de hoy se conservan: se les añade escritura, no se reemplazan.